



Shahd Usama Kassem Awad

2404004

Jana El\_sayed Mohammed

2404016

Maya Ayman Farouk

2404027

## Source Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


typedef struct node
{
    int id, height;
    char name[100];
    float gpa;
    struct node* left;
    struct node* right;
} node;


int heightt(node* n)
{
    if (n == NULL)
        return 0;
    return n->height;
```

```
}
```

```
node* newnode(int id, float gpa, char* name)
```

```
{
```

```
    node* n = malloc(sizeof(node));
```

```
    n->id = id;
```

```
    n->gpa = gpa;
```

```
    int i;
```

```
    for( i=0;name[i];i++)
```

```
        n->name[i]=name[i];
```

```
    n->name[i]='\0';
```

```
    n->left = n->right = NULL;
```

```
    n->height = 1;
```

```
    return n;
```

```
}
```

```
int max(int a, int b)
```

```
{
```

```
    return (a > b) ? a : b;
```

```
}
```

```
node* rotateright(node* y)
```

```
{
```

```
    node* x = y->left;
```

```
    node* t2 = x->right;
```

```
    x->right = y;
```

```
    y->left = t2;
```

```
    y->height = 1 + max(heightt(y->left), heightt(y->right));
```

```
    x->height = 1 + max(heightt(x->left), heightt(x->right));
```

```
    return x;
```

```
}
```

```
node* rotateleft(node* x)
```

```
{
```

```
    node* y = x->right;
```

```
    node* t2 = y->left;
```

```
    x->right = t2;
```

```
    y->left = x;
```

```
x->height = 1 + max(heightt(x->left), heightt(x->right));  
y->height = 1 + max(heightt(y->left), heightt(y->right));  
return y;  
}
```

```
int getbalance(node* root)  
{  
    if (root == NULL)  
        return 0;  
    return heightt(root->left) - heightt(root->right);  
}
```

```
node* insert(node* root, int id, float gpa, char* name)  
{  
    if (gpa < 0.0f || gpa > 4.0f)  
    {  
        printf("Error: GPA %.2f is out of range (0.0 - 4.0)\n",  
        gpa);  
        return root;  
    }
```

```
}
```

```
if (root == NULL)
```

```
    return newnode(id, gpa, name);
```

```
if (id < root->id)
```

```
    root->left = insert(root->left, id, gpa, name);
```

```
else if (id > root->id)
```

```
    root->right = insert(root->right, id, gpa, name);
```

```
else
```

```
{
```

```
    printf("Error: Student with ID %d already exists.\n",  
id);
```

```
    return root;
```

```
}
```

```
    root->height = 1 + max(heightt(root->left), heightt(root->right));
```

```
int balance = getbalance(root);
```

```
if (balance > 1 && id < root->left->id)
```

```
    return rotateright(root);
```

```
if (balance < -1 && id > root->right->id)
```

```
    return rotateleft(root);
```

```
if (balance > 1 && id > root->left->id)
```

```
{
```

```
    root->left = rotateleft(root->left);
```

```
    return rotateright(root);
```

```
}
```

```
if (balance < -1 && id < root->right->id)
```

```
{
```

```
    root->right = rotateright(root->right);
```

```
    return rotateleft(root);
```

```
}
```

```

    return root;
}

int treeSize(node* root)
{
    if (root == NULL) return 0;
    return 1 + treeSize(root->left) + treeSize(root->right);
}

int searchByID(node* root, int id)
{
    if (root == NULL)
    {
        printf("Error: Student with ID %d not found.\n", id);
        return 0;
    }

    if (root->id == id)
    {

```

```
printf("Student Found:\n");  
printf("ID   : %d\n", root->id);  
printf("Name : %s\n", root->name);  
printf("GPA  : %.2f\n", root->gpa);  
return 1;  
}
```

```
if (id < root->id)  
    return searchByID(root->left, id);
```

```
    return searchByID(root->right, id);  
}
```

```
int sameName(char* a, char* b)
```

```
{
```

```
    int i = 0;
```

```
    while (a[i] && b[i])
```

```
    {
```

```
        char c1 = a[i];
```

```
char c2 = b[i];
```

```
if (c1 >= 'A' && c1 <= 'Z')
```

```
    c1 = c1 + 32;
```

```
if (c2 >= 'A' && c2 <= 'Z')
```

```
    c2 = c2 + 32;
```

```
if (c1 != c2)
```

```
    return 0;
```

```
    i++;
```

```
}
```

```
return a[i] == '\0' && b[i] == '\0';
```

```
}
```

```
int searchByName(node* root, char* name)
```

```
{
```

```
if (root == NULL)
```

```
    return 0;
```

```
if (sameName(root->name, name))
```

```
{
```

```
    printf("Student Found:\n");
```

```
    printf("ID   : %d\n", root->id);
```

```
    printf("Name : %s\n", root->name);
```

```
    printf("GPA  : %.2f\n", root->gpa);
```

```
    return 1;
```

```
}
```

```
if (searchByName(root->left, name))
```

```
    return 1;
```

```
else
```

```
    return searchByName(root->right, name);
```

```
}
```

```
node* minValueNode(node* root)
```

```
{
```

```
node* current = root;
```

```
while (current && current->left != NULL)
```

```
    current = current->left;
```

```
return current;
```

```
}
```

```
node* deleteNode(node* root, int id)
```

```
{
```

```
    if (root == NULL)
```

```
        return NULL;
```

```
    if (id < root->id)
```

```
        root->left = deleteNode(root->left, id);
```

```
    else if (id > root->id)
```

```
        root->right = deleteNode(root->right, id);
```

```
    else
```

```
{  
  
    if (root->left == NULL || root->right == NULL)  
    {  
        node* temp = root->left ? root->left : root->right;  
  
        free(root);  
  
        return temp;  
    }  
  
    else  
    {  
        node* temp = minValueNode(root->right);  
  
        root->id = temp->id;  
        root->gpa = temp->gpa;  
  
        int i;  
        for (i = 0; temp->name[i]; i++)  
            root->name[i] = temp->name[i];  
  
        root->name[i] = '\0';  
    }  
}
```

```
        root->right = deleteNode(root->right, temp->id);
    }
}
```

```
if (root == NULL)
    return NULL;
```

```
root->height = 1 + max(heightt(root->left), heightt(root->right));
```

```
int balance = getbalance(root);
```

```
if (balance > 1 && getbalance(root->left) >= 0)
    return rotateright(root);
```

```
if (balance > 1 && getbalance(root->left) < 0)
{
    root->left = rotateleft(root->left);
```

```
    return rotateright(root);  
}
```

```
if (balance < -1 && getbalance(root->right) <= 0)  
    return rotateleft(root);
```

```
if (balance < -1 && getbalance(root->right) > 0)  
{  
    root->right = rotateright(root->right);  
    return rotateleft(root);  
}
```

```
    return root;  
}  
  
node* updateStudent(node* root, int id)  
{  
    int choice;  
    node* current = root;
```

```
while (current != NULL)  
{  
    if (id < current->id)  
        current = current->left;  
  
    else if (id > current->id)  
        current = current->right;  
  
    else  
    {  
        while (1)  
        {  
            printf("\nUpdate Menu\n");  
            printf("1. Update ID\n");  
            printf("2. Update Name\n");  
            printf("3. Update GPA\n");  
            printf("4. Exit Update Menu\n");  
            printf("Enter Your Choice: ");  
            scanf("%d", &choice);
```

```

    if (choice == 1)
    {
        int newID;

        printf("Enter New ID: ");
        scanf("%d", &newID);

        if (newID == current->id)
        {
            printf("Error: New ID is the same as current
ID.\n");

            continue;
        }

        int oldSize = treeSize(root);

        root = insert(root, newID, current->gpa, current-
>name);

```

```
if (treeSize(root) == oldSize)
```

```
    continue;
```

```
root = deleteNode(root, current->id);
```

```
current = root;
```

```
while (current != NULL)
```

```
{
```

```
    if (newID < current->id)
```

```
        current = current->left;
```

```
    else if (newID > current->id)
```

```
        current = current->right;
```

```
    else
```

```
        break;
```

```
}
```

```
printf("Student ID updated successfully.\n");  
}
```

```
else if (choice == 2)
```

```
{
```

```
char newName[100];
```

```
printf("Enter New Name: ");
```

```
getchar();
```

```
gets(newName);
```

```
int i;
```

```
for(i = 0; newName[i]; i++)
```

```
current->name[i] = newName[i];
```

```
current->name[i] = '\0';
```

```
printf("Student Name updated successfully.\n");  
}
```

```
else if (choice == 3)
```

```
{
```

```
float newGPA;
```

```
printf("Enter New GPA: ");
```

```
scanf("%f", &newGPA);
```

```
if (newGPA < 0.0f || newGPA > 4.0f)
```

```
{
```

```
printf("Error: GPA %.2f is out of range (0.0 -  
4.0)\n", newGPA);
```

```
continue;
```

```
}
```

```
current->gpa = newGPA;
```

```
        printf("Student GPA updated successfully.\n");
    }

    else if (choice == 4)
    {
        return root;
    }

    else
    {
        printf("Invalid Choice.\n");
    }
}

}

}

printf("Error: Student with ID %d not found.\n", id);

return root;
```

```
}  
  
void displayAllStudents(node* root)  
{  
    if (root == NULL)  
        return;  
  
    displayAllStudents(root->left);  
  
    printf("ID   : %d | ", root->id);  
    printf("Name : %s | ", root->name);  
    printf("GPA  : %.2f\n", root->gpa);  
  
    displayAllStudents(root->right);  
}  
  
int treeHeight(node* root)  
{  
    if (root == NULL)  
        return 0;
```

```

    int leftH = treeHeight(root->left);
    int rightH = treeHeight(root->right);

    return 1 + max(leftH, rightH);
}

void display_statical(node *root)
{
    if (root == NULL)
    {
        printf("Tree is empty.\n");
        return;
    }

    printf("Tree Height: %d\n", treeHeight(root));
    printf("Total Number of Students: %d\n", treeSize(root));
}

void minGPA(node* root, float* min)
{
    if (root == NULL)

```

**return;**

**if (root->gpa < \*min)**

**\*min = root->gpa;**

**minGPA(root->left, min);**

**minGPA(root->right, min);**

**}**

**void maxGPA(node\* root, float\* max)**

**{**

**if (root == NULL)**

**return;**

**if (root->gpa > \*max)**

**\*max = root->gpa;**

**maxGPA(root->left, max);**

**maxGPA(root->right, max);**

```
}
```

```
void sumGPA(node* root, float* sum)
```

```
{
```

```
    if (root == NULL)
```

```
        return;
```

```
    *sum += root->gpa;
```

```
    sumGPA(root->left, sum);
```

```
    sumGPA(root->right, sum);
```

```
}
```

```
void countAboveAverage(node* root, float average, int*  
count)
```

```
{
```

```
    if (root == NULL)
```

```
        return;
```

```
    if (root->gpa > average)
        (*count)++;

    countAboveAverage(root->left, average, count);
    countAboveAverage(root->right, average, count);
}

void countBelowAverage(node* root, float average, int*
count)
{
    if (root == NULL)
        return;

    if (root->gpa < average)
        (*count)++;

    countBelowAverage(root->left, average, count);
    countBelowAverage(root->right, average, count);
}
```

```
void displayGPAStatistics(node* root)
```

```
{
```

```
    if (root == NULL)
```

```
    {
```

```
        printf("Tree is empty.\n");
```

```
        return;
```

```
    }
```

```
    float min = root->gpa;
```

```
    float max = root->gpa;
```

```
    float sum = 0;
```

```
    int above = 0, below = 0;
```

```
    int total = treeSize(root);
```

```
    minGPA(root, &min);
```

```
    maxGPA(root, &max);
```

```
    sumGPA(root, &sum);
```

**float average = sum / total;**

**countAboveAverage(root, average, &above);**

**countBelowAverage(root, average, &below);**

**printf("Minimum GPA: %.2f\n", min);**

**printf("Maximum GPA: %.2f\n", max);**

**printf("Average GPA: %.2f\n", average);**

**printf("Above Avg: %d\n", above);**

**printf("Below Avg: %d\n", below);**

**}**

**node\* deleteAllStudents(node\* root)**

**{**

**if (root == NULL)**

**return NULL;**

**root->left = deleteAllStudents(root->left);**

**root->right = deleteAllStudents(root->right);**

```
free(root);
```

```
return NULL;
```

```
}
```

```
node* maxValueNode(node* root)
```

```
{
```

```
node* current = root;
```

```
while (current && current->right != NULL)
```

```
current = current->right;
```

```
return current;
```

```
}
```

```
void findPredecessorSuccessor(node* root, int id)
```

```
{
```

```
if (root == NULL)
```

```
{
```

```
    printf("The tree is empty.\n");  
    return;  
}
```

```
node* current = root;  
node* predecessor = NULL;  
node* successor = NULL;
```

```
while (current != NULL)  
{  
    if (id < current->id)  
    {  
        successor = current;  
        current = current->left;  
    }
```

```
    else if (id > current->id)  
    {  
        predecessor = current;
```

```
    current = current->right;
}

else
{
    if (current->left != NULL)
        predecessor = maxValueNode(current->left);

    if (current->right != NULL)
        successor = minValueNode(current->right);

    printf("Student Found:\n");
    printf("ID   : %d\n", current->id);
    printf("Name : %s\n", current->name);
    printf("GPA  : %.2f\n", current->gpa);

    if (predecessor != NULL)
    {
        printf("\nPredecessor:\n");
```

```
printf("ID   : %d\n", predecessor->id);  
printf("Name : %s\n", predecessor->name);  
printf("GPA  : %.2f\n", predecessor->gpa);  
}
```

**else**

```
{  
    printf("\nThis is the first record.\n");  
}
```

**if (successor != NULL)**

```
{  
    printf("\nSuccessor:\n");  
    printf("ID   : %d\n", successor->id);  
    printf("Name : %s\n", successor->name);  
    printf("GPA  : %.2f\n", successor->gpa);  
}
```

**else**

```
    {  
        printf("\nThis is the last record.\n");  
    }  
  
    return;  
}  
}  
  
printf("Error: Student with ID %d not found.\n", id);  
}  
  
int main()  
{  
    node* root = NULL;  
  
    int choice;  
  
    int id;  
  
    float gpa;  
  
    char name[100];
```

```
while (1)  
{  
  
    printf("\n===== AVL Student Management  
System =====\n");  
  
    printf("1. Insert Student\n");  
  
    printf("2. Search Student By ID\n");  
  
    printf("3. Search Student By Name\n");  
  
    printf("4. Delete Student\n");  
  
    printf("5. Update Student\n");  
  
    printf("6. Display All Students\n");  
  
    printf("7. Display Tree Statistics\n");  
  
    printf("8. Display GPA Statistics\n");  
  
    printf("9. Delete All Students\n");  
  
    printf("10. Find Predecessor and Successor\n");  
  
    printf("11. Exit\n");  
  
  
    printf("Enter Your Choice: ");  
  
    scanf("%d", &choice);
```

```
if (choice == 1)
{
    printf("Enter Student ID: ");
    scanf("%d", &id);

    printf("Enter Student full Name: ");
    getchar();
    gets(name);

    printf("Enter Student GPA: ");
    scanf("%f", &gpa);

    root = insert(root, id, gpa, name);
}

else if (choice == 2)
{
    if (root == NULL)
    {
```

```
    printf("The tree is empty.\n");  
    continue;  
}
```

```
printf("Enter Student ID: ");  
scanf("%d", &id);
```

```
    searchByID(root, id);  
}
```

```
else if (choice == 3)  
{  
    if (root == NULL)  
    {  
        printf("The tree is empty.\n");  
        continue;  
    }  
}
```

```
printf("Enter Student Name: ");
```

```
    getchar();  
    gets(name);  
  
    if (!searchByName(root, name))  
        printf("Error: Student with name %s not  
found.\n", name);  
    }  
  
    else if (choice == 4)  
    {  
        if (root == NULL)  
        {  
            printf("The tree is empty.\n");  
            continue;  
        }  
  
        printf("Enter Student ID to delete: ");  
        scanf("%d", &id);
```

```
    root = deleteNode(root, id);  
}  
  
else if (choice == 5)  
{  
    if (root == NULL)  
    {  
        printf("The tree is empty.\n");  
        continue;  
    }  
  
    printf("Enter Student ID to update: ");  
    scanf("%d", &id);  
  
    root = updateStudent(root, id);  
}  
  
else if (choice == 6)  
{
```

```
    if (root == NULL)
    {
        printf("The tree is empty.\n");
        continue;
    }

    displayAllStudents(root);
}

else if (choice == 7)
{
    if (root == NULL)
    {
        printf("The tree is empty.\n");
        continue;
    }

    display_statical(root);
}
```

```
else if (choice == 8)
{
    if (root == NULL)
    {
        printf("The tree is empty.\n");
        continue;
    }

    displayGPASTatistics(root);
}

else if (choice == 9)
{
    if (root == NULL)
    {
        printf("The tree is already empty.\n");
        continue;
    }
}
```

```
root = deleteAllStudents(root);

printf("All students deleted successfully.\n");
}

else if (choice == 10)
{
    if (root == NULL)
    {
        printf("The tree is empty.\n");
        continue;
    }
    printf("Enter Student ID: ");
    scanf("%d", &id);

    findPredecessorSuccessor(root, id);
}
```

```
    else if (choice == 11)
    {
        root = deleteAllStudents(root);

        printf("Exiting Program...\n");
        break;
    }

    else
    {
        printf("Invalid Choice.\n");
    }
}

return 0;

}
```

## Problem Description

The goal of this project is to design and implement a **Student Management System** using a **Binary Search Tree (BST)** data structure.

The system is responsible for storing and managing student records efficiently. Each student record consists of a **Student ID, Name, and GPA**, where the **Student ID is used as the primary key** to organize the data within the BST.

A Binary Search Tree is a hierarchical data structure in which each node has at most two children. For any given node, all nodes in the left subtree contain smaller IDs, while all nodes in the right subtree contain larger IDs. This property allows operations such as insertion, searching, and deletion to be performed efficiently compared to linear data structures.

The system should support essential operations including inserting new students, searching for students, updating their data, and deleting records, while maintaining the BST properties at all times.

The overall objective is to build an efficient and organized system that demonstrates the use of tree-based data structures in solving real-world data management problems

## Data Structures Used

The main data structure used in this project is an **AVL Tree**, which is a self-balancing version of the Binary Search Tree (BST).

Each node in the tree contains the following fields:

- **Student ID** (used as the key)
- **Student Name**
- **Student GPA**
- **Height** (used to maintain balance)
- **Left and Right pointers**

The AVL tree ensures that the height difference between the left and right subtrees (balance factor) is at most 1. This guarantees that all operations such as insertion, deletion, and searching are performed efficiently in  **$O(\log n)$**  time.

To maintain balance, the tree performs rotations:

- Left Rotation
- Right Rotation
- Left-Right Rotation

- Right-Left Rotation

## Complexity Analysis

This project is implemented using an **AVL Tree**, which guarantees that the tree remains balanced after every insertion and deletion. Therefore, most operations have logarithmic time complexity.

### 1. Insert a Student

- **Time Complexity:**  $O(\log n)$
- The insertion follows BST rules, then the tree is rebalanced using rotations.
- The height of the AVL tree is always  $O(\log n)$ , so insertion is efficient.

### 2. Search by ID

- **Time Complexity:**  $O(\log n)$
- Since the tree is ordered by student ID, searching follows a single path from root to leaf.

### 3. Search by Name

- **Time Complexity:**  $O(n)$
- The tree is not ordered by name, so all nodes must be traversed.

### 4. Delete a Student

- **Time Complexity:**  $O(\log n)$
- Deletion is similar to BST deletion, followed by rebalancing operations.

### 5. Update Student Information

- **Update Name or GPA:**  $O(\log n)$ 
  - First locate the student using ID, then update the values.
- **Update ID:**  $O(\log n)$ 
  - Requires deleting the old node and inserting a new one.

### 6. Display All Students

- **Time Complexity:**  $O(n)$

- Uses in-order traversal to print all students in sorted order.

## 7. Tree Statistics (Height, Total Students)

- **Time Complexity:**  $O(n)$
- Requires traversing all nodes to compute size or other statistics.

## 8. GPA Statistics (Average, Highest, Lowest)

- **Time Complexity:**  $O(n)$
- All nodes must be visited to compute these values.

## 9. Delete All Students

- **Time Complexity:**  $O(n)$
- Every node in the tree must be deleted.

## 10. Find Predecessor and Successor

- **Time Complexity:**  $O(\log n)$
- Based on BST properties, moving down the tree to find adjacent value

# Screenshots of test runs

## 4.1 Insert Operation

Shows insertion of multiple students

Includes duplicate ID rejection

GPA validation (0.0 – 4.0)

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 1
Enter Student ID: 2
Enter Student full Name: shahd
Enter Student GPA: 4
```

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 1
Enter Student ID: 2
Enter Student full Name: jana
Enter Student GPA: 4
Error: Student with ID 2 already exists.
```

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 1
Enter Student ID: 4
Enter Student full Name: mohamed
Enter Student GPA: 5
Error: GPA 5.00 is out of range (0.0 - 4.0)
```

## 4.2 Search Operations

Search by ID (found / not found cases)

Search by Name (case-insensitive)

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 2
Enter Student ID: 2
Student Found:
ID   : 2
Name : shahd
GPA  : 4.00
```

===== AVL Student Management System =====

1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit

Enter Your Choice: 2

Enter Student ID: 9

Error: Student with ID 9 not found.

===== AVL Student Management System =====

1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit

Enter Your Choice: 3

Enter Student Name: SHahD

Student Found:

ID : 2

Name : shahd

GPA : 4.00

### 4.3 Display All Students

Output shows students in ascending order (in-order traversal)

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 6
ID   : 2 | Name : shahd | GPA  : 4.00
ID   : 4 | Name : jana  | GPA  : 4.00
ID   : 22 | Name : sara  | GPA  : 3.00
```

### 4.4 Delete Operation

Before and after deleting a node

Tree remains balanced

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 6
ID   : 2 | Name : sara  | GPA  : 3.00
ID   : 4 | Name : hana  | GPA  : 2.00
ID   : 7 | Name : ali   | GPA  : 2.00
ID   : 15 | Name : mohamed | GPA  : 4.00
ID   : 19 | Name : ahmed | GPA  : 3.00
```

## 4.5 Update Operation

Updating Name, GPA, and ID

ID update handled via delete + insert

```
===== AVL Student Management System =====
```

1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit

```
Enter Your Choice: 6
```

```
ID    : 2 | Name : hana | GPA   : 3.00
```

```
ID    : 4 | Name : sara | GPA   : 4.00
```

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 5
Enter Student ID to update: 2

Update Menu
1. Update ID
2. Update Name
3. Update GPA
4. Exit Update Menu
Enter Your Choice: 1
Enter New ID: 9
Student ID updated successfully.
```

```
Update Menu
1. Update ID
2. Update Name
3. Update GPA
4. Exit Update Menu
Enter Your Choice: 1
Enter New ID: 4
Error: Student with ID 4 already exists.
```

```
Update Menu
1. Update ID
2. Update Name
3. Update GPA
4. Exit Update Menu
Enter Your Choice: 2
Enter New Name: jana
Student Name updated successfully.
```

```
Update Menu
1. Update ID
2. Update Name
3. Update GPA
4. Exit Update Menu
Enter Your Choice: 3
Enter New GPA: 4
Student GPA updated successfully.
```

```
Update Menu
1. Update ID
2. Update Name
3. Update GPA
4. Exit Update Menu
Enter Your Choice: 4

===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 6
ID   : 4 | Name : sara | GPA  : 4.00
ID   : 9 | Name : jana | GPA  : 4.00
```

## 4.6 Tree Statistics

Height of tree

Total number of students

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 7
Tree Height: 2
Total Number of Students: 2
```

## 4.7 GPA Statistics

Minimum GPA

Maximum GPA

Average GPA

Above/Below average counts

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 6
ID   : 4 | Name : sara | GPA   : 4.00
ID   : 9 | Name : jana | GPA   : 4.00
ID   : 13 | Name : ramy | GPA   : 2.00
```

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 8
Minimum GPA: 2.00
Maximum GPA: 4.00
Average GPA: 3.33
Above Avg: 2
Below Avg: 1
```

## 4.8 Predecessor and Successor

Shows previous and next student in sorted order.

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 10
Enter Student ID: 9
Student Found:
ID    : 9
Name  : jana
GPA   : 4.00

Predecessor:
ID    : 4
Name  : sara
GPA   : 4.00

Successor:
ID    : 13
Name  : ramy
GPA   : 2.00
```

#### 4.9 Delete All Students

Tree cleared successfully

Memory freed confirmation

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 9
All students deleted successfully.
```

```
===== AVL Student Management System =====
1. Insert Student
2. Search Student By ID
3. Search Student By Name
4. Delete Student
5. Update Student
6. Display All Students
7. Display Tree Statistics
8. Display GPA Statistics
9. Delete All Students
10. Find Predecessor and Successor
11. Exit
Enter Your Choice: 6
The tree is empty.
```